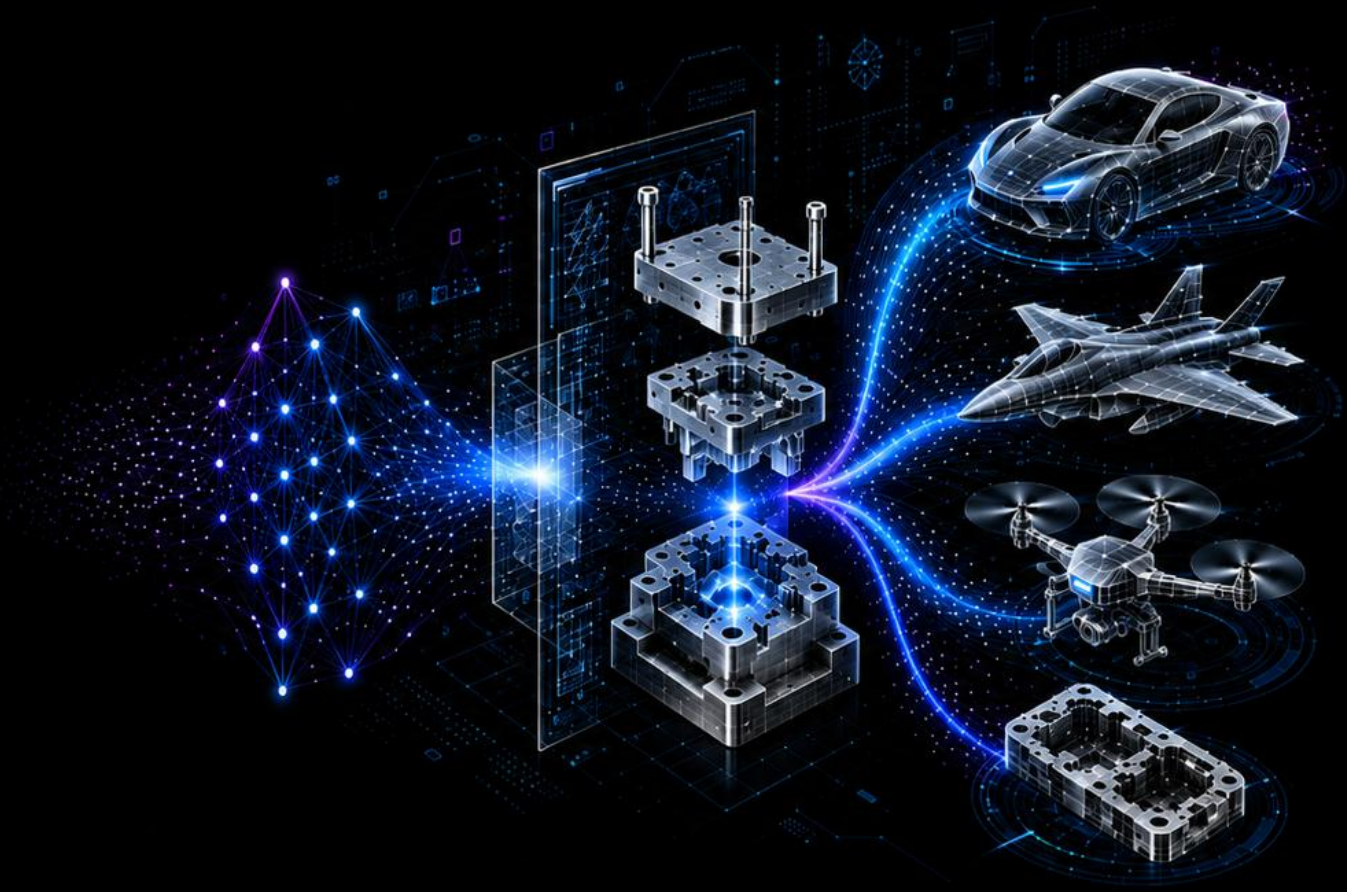
Study

# RNN / LSTM / GRU




하형진

(지도교수: 양선웅)

한양대학교 ERICA 기계공학과


Intelligent Design for Engineering Applications Lab

Part 1 Introduction
Part 2 Method
Part 3 Results & Discussion
Part 4 Conclusion

# Sequential Data

- 순차 데이터는 어떤 순서를 가진 데이터로, 순서가 변경될 경우 데이터의 특성을 잃어버리는 데이터를 말함
- 시간적 의미가 있는 경우 Temporal Sequence라고 하며, 일정한 시간차라면 Time Series라고 함

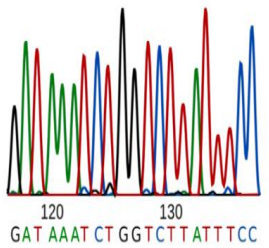
❖ e.g. “I like chocolate.”라는 문장은 이해할 수 있지만, “Like chocolate.”라는 문장은 이해할 수 없음

❖ 문장 뿐만 아니라 주가, DNA 염기서열, 날씨와 같은 Time Series Data는 모두 순차 데이터에 해당

❖ FFNN과 달리 Sequential Data를 분석하기 위한 모델은 과거 정보를 기억하는 기능이 필요함. 과거 정보를 기억하기 위해 사용하는 대표적인 방법은 이전에 사용한 데이터를 재사용하는 방법이 있음

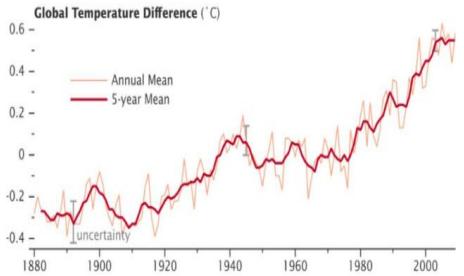
❖ 데이터를 재사용하기 위해서는 신경망 층에 순환될 필요가 있다. 이런 알고리즘을 순환 신경망(Recurrent Neural Network)이라고 부름

❖ Linear Regression, KNN, Random Forest 같은 전통적인 머신러닝 모델이나 CNN, ANN과 같은 딥러닝 모델은 데이터가 입력부터 출력까지 일방향으로만 전달되어 FFNN(Feedforward Neural Network)라고 함



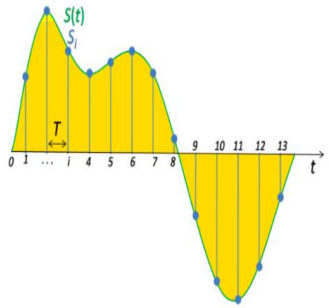

DNA 염기 서열

(Sequential Data)




세계 기온 변화

(Temporal Sequence)



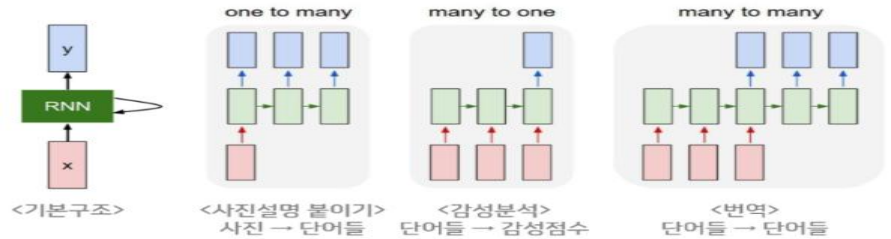

샘플링된 소리 신호

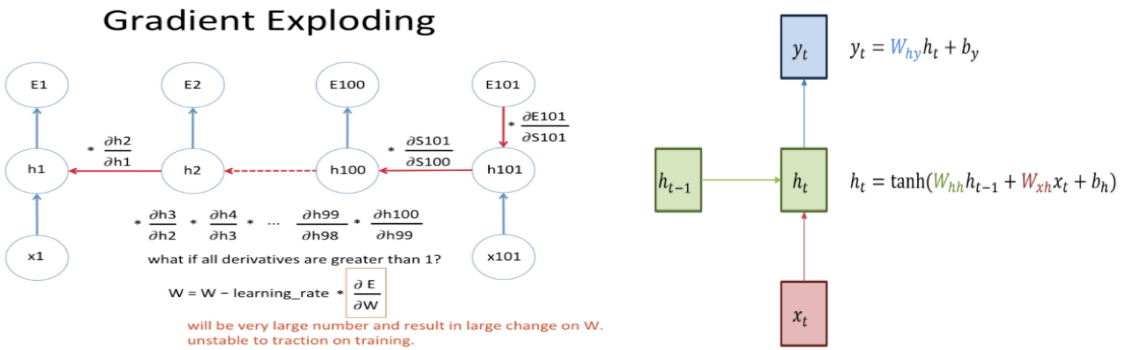
(Time Series)

Part 1 Introduction
Part 2 Method
Part 3 Results & Discussion
Part 4 Conclusion

# RNN (Recurrent Neural Network)

- RNN은 은닉층 노드들이 연결되어 이전 단계 정보를 은닉층 노드에 저장할 수 있도록 구성한 신경망




- 입력 개수, 출력 개수에 따라서 one to many, many to one, many to many로 나눠지지만, 핵심은 이전 hidden state의 아웃풋과 현시점의 인풋이 함께 연산 된다는 것
- 입력 시퀀스의 길이가 너무 길면 gradient 주는 편미분 과정에서 gradient가 사라지거나 폭발하는 문제가 생김
- 결국 다른 딥러닝 모델들과 마찬가지로 back propagation 과정을 통해 gradient를 편미분하는데, 문제는 RNN의 구조상 입력 데이터의 길이가 길어지면 에러 값 계산 후 처음 hidden state까지 back propagatio하며 gradient 주는 과정이 너무 길기 때문에 gradient 값이 연산 과정에서 작아져버려 결국 gradient가 전달이 잘 안되는데 이를 gradient vanishing이라 함
- e,g. 수학 문제를 열심히 풀고 마지막에 채점을 하는데 그 문제가 예전에 풀었던 문제라 채점을 해도 기억이 안나는 느낌. 즉, 학습이 안되는 상황

- 하지만 문제점은 many to many와 같은 구조에서 초반부에 입력된 것이 후반부에 잊혀지는 것
- 이러한 문제를 gradient vanishing이라 하고 학습에 영향을 미치는 gradient가 점점 소멸되는 것
- 따라서 예전 기억도 계속 리마인드 시켜주는 학습 방법이 필요한데 그것이 LSTM

Part 1 Introduction
Part 2 Method
Part 3 Results & Discussion
Part 4 Conclusion

# LSTM

- RNN이 오래된 것을 기억하지 못하는 기울기 소실 문제를 해결하기 위한 알고리즘
- 기억할 것과 잊을 것을 선택해 중요한 정보만 기억하는 것이 핵심

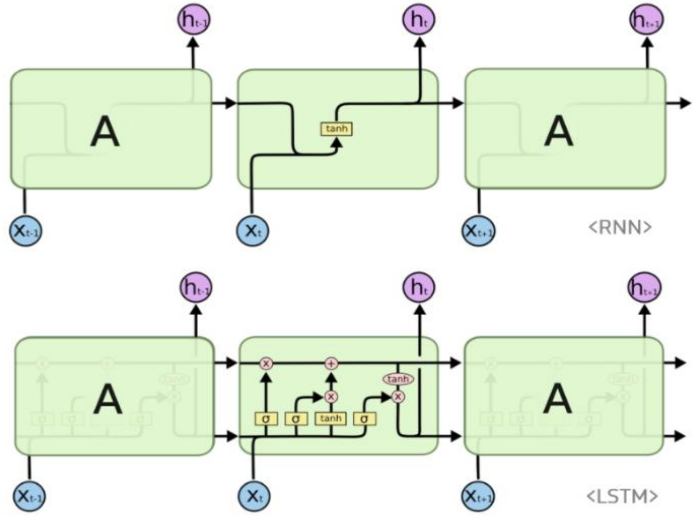
❖ RNN과 차이점은 순환되는 층이 2개, hidden state와 cell state에서 과거 정보가 순환. cell state는 hidden state와 달리 다음 층으로 전달되지 않음.

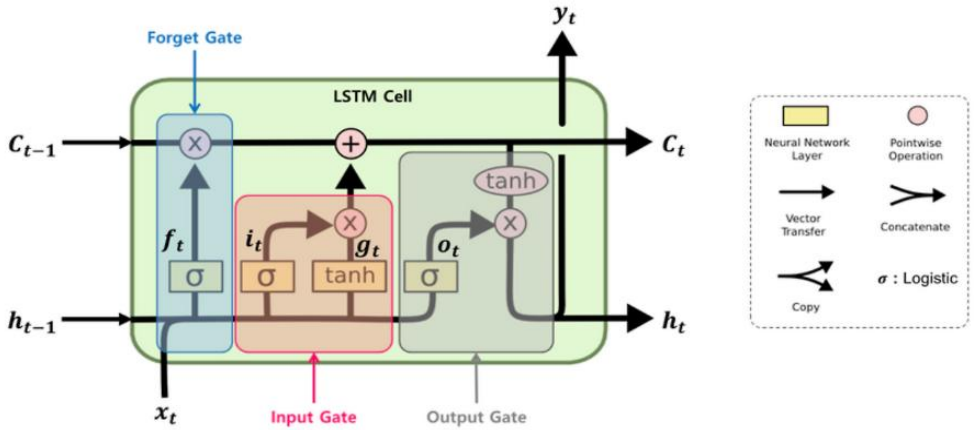
❖ LSTM은 gate를 통해서 정보를 통제, 이름대로 지나갈 수 있는 문을 만들어서 원하는 정보만 통과시키는 방법

❖ Input gate: 현재 정보를 얼마나 기억할지 결정하는 gate. 이전 층의 hidden state에는 tanh를 곱하고 새로운 정보에는 sigmoid 활성함수를 곱함. 새로운 특성에 sigmoid를 곱하기 때문에 0~1 사이의 결과 값으로 새로운 정보를 얼마나 사용할지 결정

❖ Forget gate: 정보를 얼마나 잊어버릴지 결정하는 gate. 입력과 hidden state를 서로 다른 가중치를 곱한 다음 sigmoid 활성 함수를 통과시킴. 0~1 사이의 결과값을 얻기 때문에 얼마나 정보를 잊을지 결정할 수 있음. 이전 타임스텝의 cell state와 곱하여 새로운 cell state를 만듦

❖ Output gate: 다음 층으로 전달할 hidden state를 만드는 gate. 이전 hidden state 값과 입력 값에 각각 다른 가중치를 곱하고 sigmoid 활성 함수를 통과시킴. 그 출력 값과 현재 cell state 값을 tanh 활성함수에 통과시킨 값을 서로 곱해서 hidden state를 만듦



Part 1 Introduction
Part 2 Method
Part 3 Results & Discussion
Part 4 Conclusion

# GRU

- LSTM을 개선한 모델, LSTM보다 파라미터가 더욱 적어 연산 비용도 적고 모델도 간단해 학습 속도가 더 빠르지만 비슷한 성능을 내는 모델, Forget gate와 Input gate를 합쳐 Update gate를 만들고 Reset gate를 추가

❖ Update gate

다음 상태를 따라 전달해야 하는 이전 정보의 양을 결정하는 역할

Forget gate와 Input gate를 합한 gate이고 입력 값과 hidden state 값을 각각 가중치에 곱하고 sigmoid 함수에 통과시켜 forget gate로 사용. 이 값을 1에서 빼 Input gate로 사용

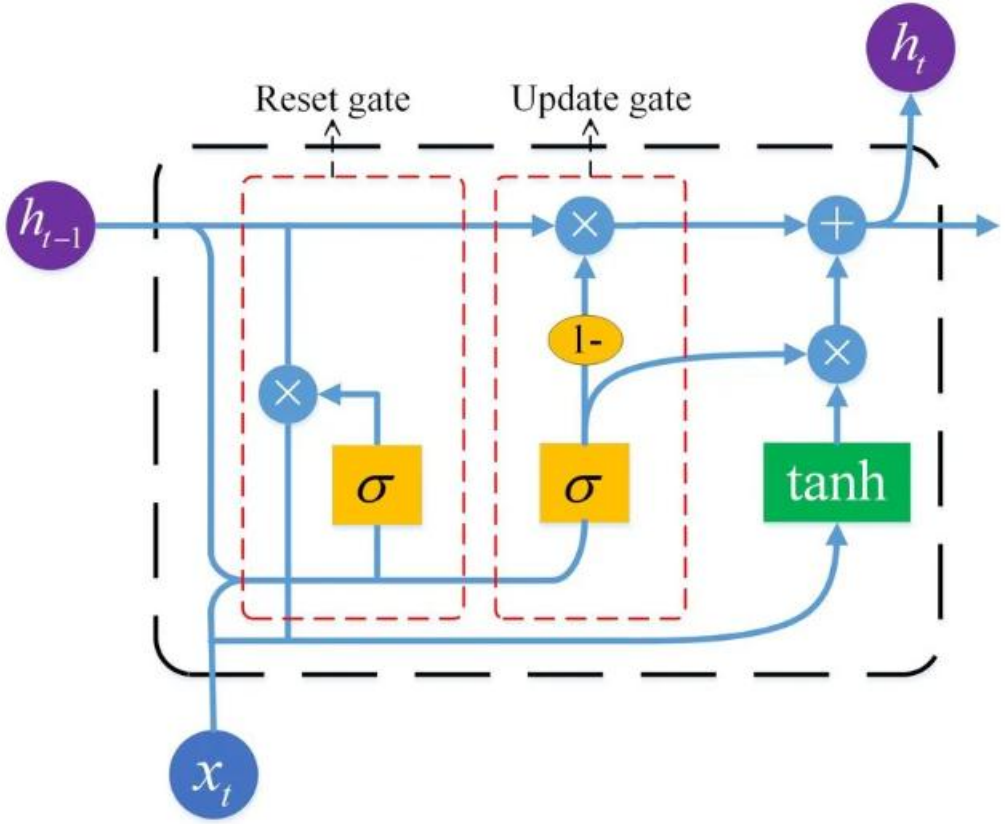
❖ Reset gate

모델에서 무시해야 할 과거 정보의 양을 결정하는데 사용.

(이전 셀 상태가 중요한지 여부를 결정)

Reset gate가 작동하여 과거 시간 단계의 관련 정보를 새 메모리 콘텐츠에 저장, 입력 벡터와 hidden state에 가중치를 곱함.

Reset gate와 이전에 hidden state 배수 사이의 요소별 곱셈을 계산, 위 단계를 종합한 후에 비선형 활성화 함수가 적용되고 다음 시퀀스가 생성



Part 1 Introduction
Part 2 Method
Part 3 Results & Discussion
Part 4 Conclusion

# RNN

$\mathbf{x}_t$ : input vector $(m \times 1)$.
$\mathbf{h}_t$ : hidden layer vector $(n \times 1)$.
$\mathbf{o}_t$ : output vector $(n \times 1)$.
$\mathbf{b}_h$ : bias vector $(n \times 1)$.
U, W : parameter matrices $(n \times m)$.
$V$ : parameter matrix $(n \times n)$.
$\sigma_h, \sigma_y$ : activation functions.

은닉층 $h_t = tanh(W_x x_t + W_h h_{t-1} + b)$
출력층 $y_t = f(W_y h_t + b)$

## Feed-Forward

$$\mathbf{h}_t = \sigma_h(\mathbf{z}_t) = \sigma_h(\mathrm{U}_h \mathbf{x}_t + V_h \mathbf{h}_{t-1} + \mathbf{b}_h)$$

$$y_t = \sigma_y(a_t) = \sigma_y(\mathrm{W}_y \mathbf{h}_t + \mathbf{b}_h)$$

## Backpropagation

$$\left[\Pi_t = \frac{\partial E_t}{\partial \mathbf{o}_t}\frac{\partial \mathbf{o}_t}{\partial \mathbf{h}_t} + \frac{\partial h_{t+1}}{\partial \mathbf{h}_t}\Pi_{t+1}\right]$$

$$\left[\beta_t^U = \beta_{t+1}^U + \Pi_t \frac{\partial \mathbf{h}_t}{\partial \mathrm{U}_t}\right]$$

$$\left[\beta_t^V = \beta_{t+1}^V + \Pi_t \frac{\partial \mathbf{h}_t}{\partial V_t}\right]$$

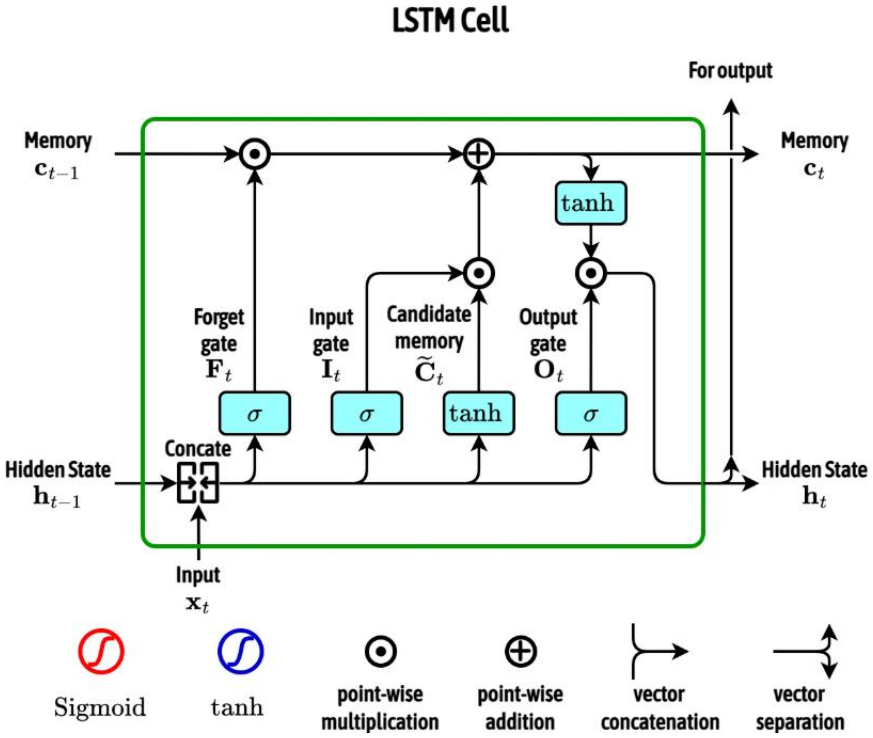
$$\left[\beta_t^W = \beta_{t+1}^W + \frac{\partial E_t}{\partial \mathbf{o}_t}\frac{\partial \mathbf{o}_t}{\partial \mathrm{W}_t}\right]$$

$$\left[\frac{\partial E}{\partial X} \equiv \beta_0^x\right]$$

Part 1 Introduction
Part 2 Method
Part 3 Results & Discussion
Part 4 Conclusion

# LSTM

$\mathbf{h}_t, C_t$ : hidden layer vectors.
$\mathbf{x}_t$ : input vector.
$b_f, b_i, b_c, b_o$ : bias vector.
$W_f, W_i, W_c, W_o$ : parameter matrices.
$\sigma, \tanh$ : activation functions.




## Feed-Forward

$$\mathbf{F}_t = \sigma\left(\mathrm{W}_f \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f\right)$$
$$\mathbf{I}_t = \sigma(\mathrm{W}_i \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i)$$
$$\mathbf{O}_t = \sigma(\mathrm{W}_o \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o)$$
$$\tilde{\mathbf{C}}_t = \tanh(\mathrm{W}_c \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c)$$
$$\mathbf{c}_t = \mathbf{F}_t \odot \mathbf{C}_{t-1} + \mathbf{I}_t \odot \tilde{\mathbf{C}}_t$$
$$\mathbf{h}_t = \mathbf{O}_t \odot \tanh(\mathbf{C}_t)$$

## Backpropagation

$$\frac{\partial C_{t+1}}{\partial \mathbf{h}_t} = \frac{\partial C_{t+1}}{\partial \tilde{C}_{t+1}} \frac{\partial \tilde{C}_{t+1}}{\partial \mathbf{h}_t} + \frac{\partial C_{t+1}}{\partial f_{t+1}} \frac{\partial f_{t+1}}{\partial \mathbf{h}_t} + \frac{\partial C_{t+1}}{\partial t_{t+1}} \frac{\partial i_{t+1}}{\partial \mathbf{h}_t}$$

$$\frac{\partial C_{t+1}}{\partial C_t}$$

$$\frac{\partial h_{t+1}}{\partial C_t} = \frac{\partial h_{t+1}}{\partial C_{t+1}} \frac{\partial C_{t+1}}{\partial C_t}$$

$$\frac{\partial h_{t+1}}{\partial \mathbf{h}_t} = \frac{\partial h_{t+1}}{\partial C_{t+1}} \frac{\partial C_{t+1}}{\partial \mathbf{h}_t} + \frac{\partial h_{t+1}}{\partial o_{t+1}} \frac{\partial o_{t+1}}{\partial \mathbf{h}_t}$$

$$\left[\Pi_t = \frac{\partial E_t}{\partial \mathbf{h}_t} + \frac{\partial h_{t+1}}{\partial \mathbf{h}_t}\Pi_{t+1} + \frac{\partial C_{t+1}}{\partial \mathbf{h}_t}\mathcal{T}_{t+1}\right]$$

$$\left[\mathcal{T}_t = \frac{\partial E_t}{\partial \mathbf{h}_t}\frac{\partial E_t}{\partial C_t} + \frac{\partial h_{t+1}}{\partial C_t}\Pi_{t+1} + \frac{\partial C_{t+1}}{\partial C_t}\mathcal{T}_{t+1}\right]$$

$$\left[\beta_t^f = \beta_{t+1}^f + \frac{\partial C_t}{\partial f_t}\frac{\partial f_t}{\partial W_t^f}\left(\frac{\partial \mathbf{h}_t}{\partial C_t}\Pi_t + \mathcal{T}_t\right)\right]$$

$$\left[\beta_t^i = \beta_{t+1}^i + \frac{\partial C_t}{\partial i_t}\frac{\partial i_t}{\partial W_t^i}\left(\frac{\partial \mathbf{h}_t}{\partial C_t}\Pi_t + \mathcal{T}_t\right)\right]$$

$$\left[\beta_t^c = \beta_{t+1}^c + \frac{\partial C_t}{\partial \tilde{C}_t}\frac{\partial \tilde{C}_t}{\partial W_t^c}\left(\frac{\partial \mathbf{h}_t}{\partial C_t}\Pi_t + \mathcal{T}_t\right)\right]$$

$$\left[\beta_t^o = \beta_{t+1}^o + \frac{\partial \mathbf{h}_t}{\partial \mathbf{o}_t}\frac{\partial \mathbf{o}_t}{\partial W_t^o}(\Pi_t)\right]$$

Part 1 Introduction
Part 2 Method
Part 3 Results & Discussion
Part 4 Conclusion

# GRU

$\mathbf{h}_t$ : hidden layer vectors.
$\mathbf{x}_t$ : input vector.
$b_z, b_r, \mathbf{b}_h$ : bias vector.
$W_z, W_r, W_h$ : parameter matrices.
$\sigma, \tanh$ : activation functions.

## Feed-Forward

$$[z_t = \sigma(W_z \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_z)]$$

$$[r_t = \sigma(W_r \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_r)]$$

$$\left[\tilde{\mathbf{h}}_t = \tanh(W_h \cdot [r_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_h)\right]$$

$$\left[\mathbf{h}_t = (1 - z_t) \odot \mathbf{h}_{t-1} + z_t \odot \tilde{\mathbf{h}}_t\right]$$

## Backpropagation

$$\left[\frac{\partial h_{t+1}}{\partial \mathbf{h}_t} = \frac{\partial h_{t+1}}{\partial \mathbf{h}_t} + \frac{\partial h_{t+1}}{\partial z_{t+1}}\frac{\partial z_{t+1}}{\partial \mathbf{h}_t} + \frac{\partial h_{t+1}}{\partial \tilde{h}_{t+1}}\left(\frac{\partial \tilde{h}_{t+1}}{\partial \mathbf{h}_t} + \frac{\partial \tilde{h}_{t+1}}{\partial r_{t+1}}\frac{\partial r_{t+1}}{\partial \mathbf{h}_t}\right)\right]$$

$$\left[\Pi_t = \frac{\partial E_t}{\partial \mathbf{h}_t} + \frac{\partial h_{t+1}}{\partial \mathbf{h}_t}\Pi_{t+1}\right]$$

$$\left[\beta_t^z = \beta_{t+1}^z + \frac{\partial \mathbf{h}_t}{\partial z_t}\frac{\partial z_t}{\partial W_t^z}\Pi_t\right]$$

$$\left[\beta_t^r = \beta_{t+1}^r + \frac{\partial \mathbf{h}_t}{\partial \tilde{h}_t}\frac{\partial \tilde{h}_t}{\partial r_t}\frac{\partial r_t}{\partial W_t^r}\Pi_t\right]$$

$$\left[\beta_t^h = \beta_{t+1}^h + \frac{\partial \mathbf{h}_t}{\partial \tilde{h}_t}\frac{\partial \tilde{h}_t}{\partial W_t^h}\Pi_t\right]$$

Part 1 Introduction
Part 2 Method
Part 3 Results & Discussion
Part 4 Conclusion

# RNN, LSTM, GRU

## Results

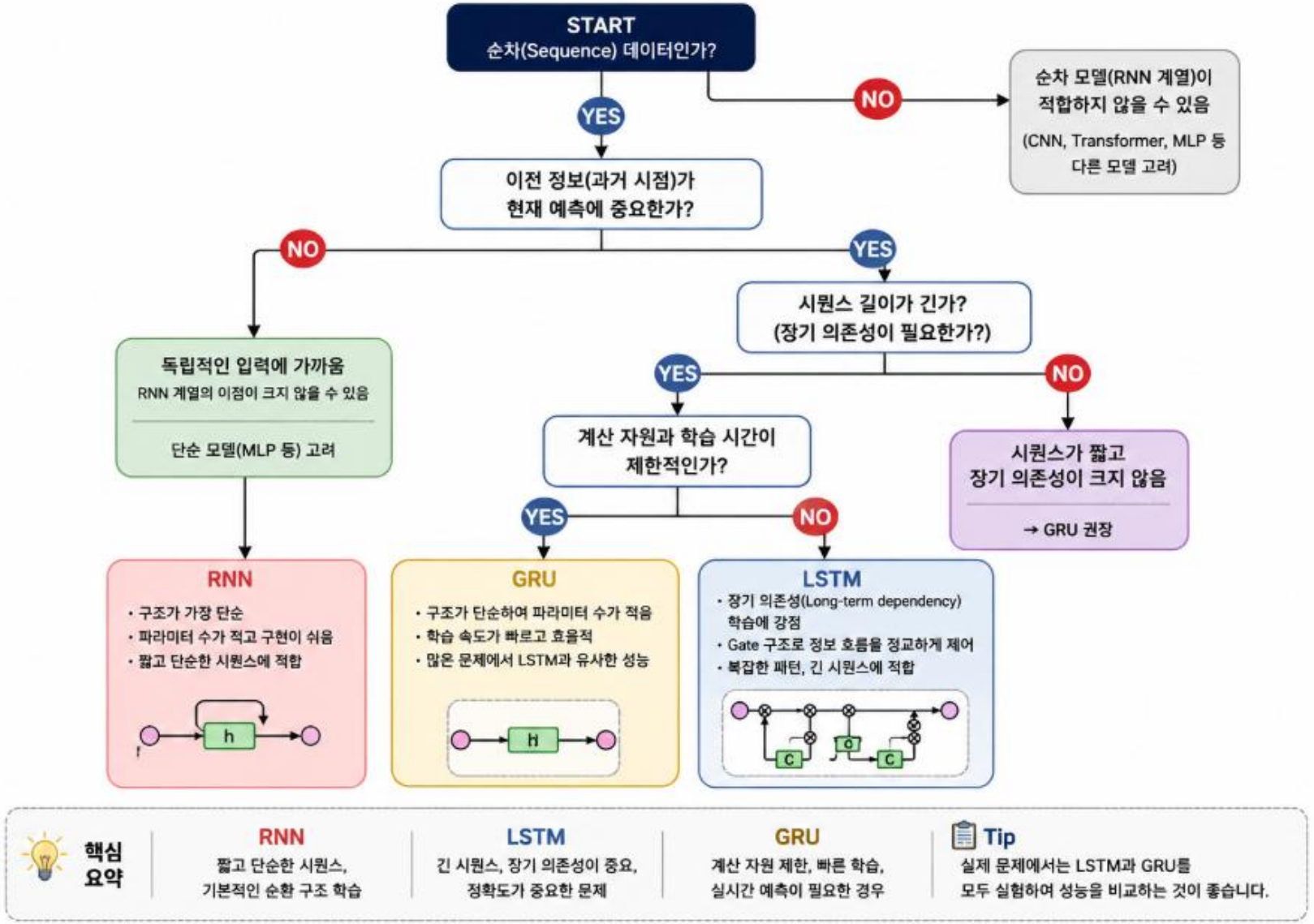
- ❖ RNN은 순차 데이터를 처음으로 효과적으로 처리한 신경망
- ❖ 하지만 긴 시퀀스에서는 Vanishing Gradient 문제 발생
- ❖ LSTM은 Memory cell과 Gate 구조를 통해 장기 의존성 해결
- ❖ GRU는 LSTM을 단순화하여 비슷한 성능을 더 빠르게 구현

## Discussion

- ❖ RNN을 배우는 이유?
  - 모든 순환신경망의 기본 구조
  - LSTM, GRU 기반 개념
  - Hidden state 전달 원리를 이해하기 위해 필수
- ❖ LSTM이 항상 좋은지?
  - 데이터가 길수록 LSTM이 유리
  - 데이터가 짧으면 GRU가 더 빠르고 성능도 비슷
- ❖ 최근 연구 동향에서 현재 NLP에서는 Transformer가 RNN 계열을 대부분 대체하지만 시계열, 센서 데이터, 음성, Embedded AI에서는 여전히 LSTM, GRU가 자주 사용된다.

| Feature | RNN | LSTM | GRU |
|---|---|---|---|
| Gradient Flow | Poor | Good | Good |
| Long-term Dependencies | Difficult | Excellent | Excellent |
| Parameters | Fewer | More | Medium |
| Training Speed | Fastest | Slow | Medium |
| Complexity | Simple | Complex | Medium |
| Memory | Low | High | Medium |

Part 1 Introduction
Part 2 Method
Part 3 Results & Discussion
Part 4 Conclusion



# Any Questions?


이름 하 형 진

소속 한양대학교 기계공학과

이메일 broha@hanyang.ac.kr


IDEA Intelligent Design for Engineering Applications Lab

